# Watchdogs, Guardrails, Locks and Controls

The mechanisms that bring a node back when it stalls, stop it from doing harm, keep two actions from colliding, and let an operator change things safely.

WATCHDOGS GUARDRAILS LOCKS CONTROLS

SUMMARY · READ-ONLY REVIEW

Source: the tool files on branch ccr-9d4fe19f-ssud83. Values are from the code unless a row is marked (manual).

# What counts as a control

Four kinds of control, grouped by what they protect against.

| KIND | WHAT IT DOES | EXAMPLE |
|---|---|---|
| **Watchdog** | Watches a service, link, tunnel or supply and acts on its own. | wifimon's voltage watchdog shuts the node down after sustained low voltage. |
| **Guardrail** | Checks an action before it runs and refuses or warns. | 44helper refuses to open a loopback-only port in any firewall zone. |
| **Lock** | Stops two actions, or two people, from working on the same thing. | Five wrong passwords lock an address out for a while. |
| **Control** | A setting or button an operator uses to pause, change or undo. | Quiet mode pauses services before a big install. |

## 1.1 How to read the tables

- Values are the ones in the code, with the file's constant name where it helps.
- Rows marked **(manual)** describe behaviour from the user manual that was not re-checked in the code for this summary.
- Times are in seconds unless stated otherwise.
- Tool names follow the repository (wifimon, 44helper, nodecontrol, nodetalker, sysmon, instmon). Text uses Node Controller for the Dashboard's own name.

# The guard dogs

Each watchdog watches one thing and has one job. Most act only when a check has failed several times in a row, so a single blip does nothing.

## 2.1 Service watchdogs (systemd)

Every tool runs as a systemd service with `Restart=always`. Each one also sends a heartbeat to systemd. If the heartbeat stops, systemd restarts the service.

| SERVICE | HEARTBEAT WINDOW | WHERE SET |
|---|---|---|
| **nodecontrol (Dashboard), pi02w** | 30 s | nodecontrol_pi02w, unit file (line 176) |
| **sysmon, pi02w** | 30 s | sysmon_pi02w, unit file |
| **44helper** | 60 s | 44helper, unit file (line 539) |
| **wifimon** | 60 s | wifimon, unit file (line 336) |
| **nodetalker (Node Talker)** | 60 s | nodetalker, unit file (line 197) |
| **amp_nodetalker, noderecorder, instmon** | 60 s | each tool's unit file |

The installer no longer installs a separate watchdog script. The Dashboard's own service covers that job, and the installer says so. An old copy is removed on install.

## 2.2 Hardware watchdog (wifimon)

- On by default (`HW_WATCHDOG_ON`), with a 15 s limit (`HW_WATCHDOG_SECS`).
- Set through systemd's `RuntimeWatchdogSec=15s`, using `/dev/watchdog0` or `/dev/watchdog`.
- On a Pi without a hardware watchdog device, the page says there is nothing to turn on.
- wifimon also has a loop-stall limit of 180 s (`LOOP_STALL_SECS`).

## 2.3 WiFi watchdog (wifimon)

| SETTING | VALUE | MEANING |
|---|---|---|
| **Check interval** | 5 s | How often the connection is tested. |
| **Misses before "lost"** | 3 | Three failed checks in a row before the link counts as lost. |
| **No-connection limit** | 180 s | How long the link can stay down before the loss action runs. |
| **Reconnect interval** | 15 s | Time between reconnect attempts. Each attempt is verified for 15 s. |
| **NetworkManager head start** | 30 s, then up to 45 s busy | Gives NetworkManager time to reconnect before wifimon acts. |
| **Stuck link** | 60 s | The interface reports up but there is no connection for 60 s. This is treated as a stuck link and sends wifimon to its recovery steps. |
| **Boot grace** | 600 s | In the first 10 minutes after boot, a missing network does not trigger the loss action, because the router may still be starting. |

| | | |
|---|---|---|
| **Change grace** | 180 s | After a network change, the shutdown timer pauses while the new link settles. |

The loss action is set by `network_loss_action`. The code default is **reboot**, at most 3 times in 6 hours (`NET_REBOOT_LIMIT`, `NET_REBOOT_WINDOW_SECS`). After the third reboot in the window, the node stays up and keeps trying to reconnect. The other choice is power off. See Part 6, point 1.

On a fresh install, both watchdogs are written as off. The first login asks whether to turn them on (see Part 6, point 2).

## 2.4 Voltage watchdog (wifimon)

- Counts readings from 0.50 V up to 0.90 V (`LOW_VOLTAGE_THRESHOLD`) as low voltage. Readings outside that range are not counted.
- Shuts the node down after the low reading lasts 30 s (`LOW_VOLTAGE_SHUTDOWN_SECS`). Shutdown is power off, not reboot.
- The page shows the two counters as they run. When a problem clears, the counter resets.

## 2.5 Tunnel watchdog (wifimon, for the 44Net tunnel)

Watches the WireGuard tunnel `wg0` that 44helper sets up. It is on by default (`tunnel_guard` defaults to enabled).

| SETTING | VALUE | MEANING |
|---|---|---|
| **Check interval** | 60 s | How often the handshake is checked. |
| **Stale handshake** | 180 s | A handshake older than this counts as bad. |
| **Probe** | ping 44.0.0.1, then wait 6 s | Sent before the check, to wake a quiet tunnel. |
| **Bad checks before restart** | 3 | Consecutive bad checks before the tunnel is restarted. |
| **Wait for handshake after restart** | 60 s | Time allowed for a new handshake. |
| **Restart timeout** | 240 s | A restart may take this long, because the clock wait alone can take 90 s. |
| **Back-off after failed restarts** | 0, 10 min, 30 min, 1 h | Waiting time between attempts. |
| **Maximum attempts** | 6 per boot | After that the watchdog shows "Gave up for this boot". |
| **Boot delay** | 300 s | Time after boot before the watchdog starts acting. |

On a handshake failure, the watchdog restarts the tunnel unit. When the handshake returns, it can restart netcheck and the bridges that depend on the tunnel (the restart_dv setting, on by default). While a half-tunnel or exceptions test is armed in 44helper (its state files under `/run/44helper-half` and `/run/44helper-fullx` show steps or an arm time), the watchdog stays out of the way. In the code, the tunnel watchdog never reboots or powers the node off. The manual says the same (manual). Its state is in `/run/asl_dvs/tunnel_guard.json`, and every action is written to the Activity log.

## 2.6 Link watchdogs (Node Controller and Node Talker)

| WATCHDOG | WHERE | WHAT IT DOES |
|---|---|---|
| **Bridge reaper** | nodecontrol, `_bridge_reaper` | A bridge node that stays linked for two polls while the current tab does not own it is dropped. 8 s cooldown per node. |
| **Bridge watchdog** | nodecontrol, `_bridge_watchdog` | When the owned bridge drops out for two polls, the node links it again. 8 s cooldown. |
| **Node Talker guard** | nodetalker, `guard_tick` | Checks every 5 s. A bridge link not in use is dropped after two polls. If the bridge the Node Talker set up is down, it is linked again. More than one public link drops the extra favorites, and keeps one. |

The Node Talker guard stays out when the Dashboard is running the node itself, so the two do not fight over the same node. Drops are capped at 16 links per pass, with 0.25 s pacing. The Node Talker's transmit node (1921) is never dropped by the guard.

# Checks before an action

A guardrail either blocks the action or warns and lets the operator decide. Blocks are for things that would break the node or leave it open to the wrong people.

## 3.1 Firewall port guardrail (44helper)

| PORT ROLE | RESULT | WHY |
|---|---|---|
| **Loopback-only (a local link no remote peer needs)** | **Blocked** in every zone | Opening it adds exposure with no benefit. |
| **Management (Dashboard, AMI, Cockpit)** | **Warning**, only sensible on a LAN zone | Risky if reachable from outside. Never on the 44Net tunnel zone. |
| **Anything else** | OK | No guardrail message. |

The check runs when a port is added, and it also runs as the operator types. Adding a port to a zone that firewalld does not have is refused (`_fw_zone_guard`).

## 3.2 WireGuard and AllStar zone guardrail (44helper)

The check `check_allstarlink_zone_guardrail` warns when `wg0` is attached to the ASL3 image's own `allstarlink` firewall zone, instead of 44helper's zone. The Full and Split Tunnel tabs show the result as a guardrail line (manual for the page layout; the check function is in the code).

## 3.3 Safe-change steps for the tunnel (44helper, manual)

1. Use **Add (runtime test)** before a permanent change. The port is open only until the next reboot or an undo.
2. Changes can be put back by a timer, **Undo now**, or a reboot.
3. **Make permanent** stays locked until the earlier steps are done and every check is green or grey and under 15 minutes old.

## 3.4 Network change safety (wifimon)

| SETTING | VALUE | MEANING |
|---|---|---|
| **New network test** | 30 s | A newly joined network is tested before it is kept. |
| **Fallback test** | 20 s | Time allowed for the network to fall back to a known one. |
| **Undo window** | 60 s | A change can be undone within this time. |
| **Change grace** | 180 s | The shutdown timer pauses while the new link settles. |

A pending change is recorded in `/run/wifimon/pending_change.json`.

## 3.5 Transmit guard and limits (Node Talker)

| SETTING | RANGE OR VALUE | MEANING |
|---|---|---|
| **Push-to-talk guard** | 30, 60, 90, 120, 180 s (default 60) | Used by the page in tap mode (and with a headset key): it unkeys by itself after this time, never beyond the transmit time-out. With 10 s left it flashes, beeps twice and vibrates. |
| **Tap debounce** | 500 ms | Taps closer together than this are ignored. |
| **Wait after receive** | 4 s (off by default) | When on, push-to-talk is locked for 4 s after the node receives audio. |
| **Transmit time-out** | 30 to 600 s (default 120) | The node's own limit on any one transmission. |
| **Don't transmit while busy** | On or off; 1.0 s grace | When on, the node refuses to key while other audio is active. A transmission that is already running is not affected. |

Settings outside the allowed values are refused. For example, a guard value other than the five listed raises an error when it is read.

## 3.6 Link actions (Node Controller)

- ASL and EchoLink connects refuse bridge slots. The local API refuses to disconnect them.
- A connect first drops every other link (up to 16 per pass). The Node Talker transmit node is kept.
- A link that belongs to the dashboard's own tab is never treated as foreign.

## 3.7 Config checks (every tool)

Settings are checked when they are saved. Values that could break a tool are refused. Files are written with `atomic_write`, so a power cut leaves the old file, not a half-written one (see the Common Functions Sheet).

# Keeping actions and people apart

## 4.1 Login lockout, by tool

Each tool has its own login, and the rules are not the same. The manual says "the other tools have similar limits". The code shows three different rules.

| TOOL | FAILURES ALLOWED | LOCKOUT | SESSION |
|---|---|---|---|
| **Dashboard (nodecontrol) and SysMon** | 5 in 5 min | 15 min, fixed. Applies to both tools. | 12 h session, one shared session file in `/run/asl_dvs` |
| **44helper** | 5 in 5 min | 15 min, fixed | 2 h session. The login cookie is set to last up to 365 days. |
| **Node Talker (and Node Recorder, same code)** | 5 | From the fifth failure, 5 s, doubling with each failure, capped at 300 s. Failure history forgotten after 1 h. | 12 h |
| **instmon, pibrary, Node Recorder** | 5 | The first four failures carry no wait. From the fifth, 5 s, doubling, capped at 300 s | 12 h |
| **wifimon** | 5 | From the fifth failure, 5 s, doubling, capped at 300 s | 12 h. At most 16 connections, 8 from one address. |

Node Talker also counts failed logins from the internet separately: 20 failures in 10 minutes make internet logins wait until that window ends.

Sessions and the shared file: `/run` is cleared at reboot, so a reboot logs everyone out. Stored sessions are written under a file lock.

## 4.2 Link action lock (Node Controller)

- All connect, disconnect and tab-switch actions share one reentrant lock.
- An action waits up to 2 s for the lock. If it still cannot get it, it is refused with "Busy — another connect, disconnect or tab switch is still running".
- The bridge reaper only tries the lock and skips its pass if it is busy.

## 4.3 Transmit lock (Node Talker)

The tune lock is taken without waiting. The guard skips its work if a transmit or tune is already using the lock. The busy lock refuses a transmission while another source is active, as described in Part 3.5.

## 4.4 Package manager lock (instmon, 44helper)

- Quiet mode is refused while an apt or dpkg lock is held. instmon checks four lock files, and 44helper checks the same four.
- The message names the lock file, and says to try again once apt or dpkg has finished.

## 4.5 Terminal (44helper)

- Only one terminal can be open at a time. A second one is refused with a "one at a time" message.
- A terminal closes itself after 10 minutes with no typing.
- The Asterisk CLI console blocks the `!` shell escape and Ctrl-Z (manual, not re-checked in the code).

## 4.6 Masked services and tunnel kind (44helper, SysMon, manual)

- A masked service cannot start, even by hand, until it is unmasked. The install buttons on a tab stay locked while a service that tab needs is masked.
- If one kind of tunnel (full or split) is installed, the other kind's steps are locked until the operator presses **Acknowledge and unlock**, so a working tunnel is not overwritten by accident.
- SysMon locks the modules a feature needs (shown with a lock icon) and the Dashboard's own phone block in `rpt.conf` and `modules.conf`.

# Things an operator can change

## 5.1 Pause and resume

| CONTROL | WHERE | WHAT IT DOES |
|---|---|---|
| **Quiet mode** | 44helper, instmon | Pauses the services that compete for the SD card, in tiers, before a big install or upgrade. Each stop waits up to 150 s. Watchdogs and update timers are paused too. Turning it on is refused while an apt or dpkg lock is held (manual for the tier order). |
| **Quiet System / Restore** | instmon (code); also on other pages (manual) | Pauses SysMon, the Dashboard, 44helper, Node Talker, Node Recorder, Pibrary and any watchdog timer, and restores itself after 30 minutes (INSTMON_QUIET_AUTO_RESTORE_SEC) or when Restore is pressed. The radio, Asterisk, the bridges, Allmon3, SSH and wifimon stay up. Refused while a disk-image job runs. |
| **Node reboot schedule** | 44helper (manual) | Reboots the node 3 s after the schedule fires, with `systemd-run`. |

## 5.2 Link controls (Node Controller)

| CONTROL | WHAT IT DOES |
|---|---|
| **Disconnect current, disconnect one, disconnect all** | Per-link drops. See the Connect and Disconnect Review. |
| **Emergency reset** | Drops every link, then restarts the analog bridge, the MMDVM bridge and Asterisk. Of the disconnect actions, this is the only one that restarts Asterisk. |
| **Node Talker local API** | Accepts connect and disconnect from Node Talker, only from the loopback address and with the local key. |

## 5.3 Service controls (SysMon, manual)

- **Start, Stop, Restart** for each service.
- **Reset Failed** clears a "failed" mark so the service can start cleanly.
- **Mask and Unmask** block or allow a service from starting. Mask asks first.
- Each action asks first where a stop could cut the node off.

## 5.4 Firewall controls (44helper, shared functions)

- Open and close ports through the shared `open_firewall_port` and `close_firewall_port` functions (see the Common Functions Sheet).
- Enable or disable ufw. Enabling it blocks everything with no allow rule. It asks first, and warns if no rule exists for the SSH port (manual).

## 5.5 WiFi and watchdog settings (wifimon)

| CONTROL | WHAT IT DOES |
|---|---|
| **WiFi watchdog and voltage watchdog switches** | Turn each watchdog on or off. Both start off on a fresh install. |
| **Hardware watchdog switch** | Turns the 15 s hardware watchdog on or off. |
| **Network loss action** | Reboot (default) or power off, when the link is lost. |
| **Tunnel watchdog switch** | On by default. Turn it off while working on the tunnel by hand, because a restart can drop an SSH or Cockpit session from outside. |
| **Keep adapter on, off, or Auto** | Keeps a Wi-Fi adapter on or off. Keep off writes a NetworkManager rule for it (manual). |
| **Hotspot** | Starts a hotspot. The node leaves its current WiFi while it runs (manual). |
| **Reset to defaults** | Puts every wifimon setting back to its default. |

## 5.6 Evidence after an event

- Each watchdog shutdown writes a shutdown report and a state file (`/run/wifimon/shutdown_state.json`). The newest 10 reports are kept, each up to 256 KB.
- A shutdown notice appears on SysMon and the Dashboard, reading "wifimon — Node Shutting Down".
- The Activity log of each tool records its watchdog actions.
- Restore points (44helper, Config tab) let the operator roll back a config change (manual).

# Where the manual and code differ

Found while checking this summary. The book should follow the code, or the code should be changed to match the manual.

## 6.1 Network loss: reboot, not shut down

The manual says the WiFi watchdog "shuts the Pi down after 3 minutes with no connection". The code's default is **reboot**, at most 3 times in 6 hours. After that, the node stays up and keeps trying. Power off is the other choice. The Dashboard's hint text says "Default Reboot", which matches the code. The manual is out of date here.

## 6.2 Watchdogs start off on a fresh install

The constants in the code say both watchdogs are on. But a fresh install writes them as off, and the first login asks whether to turn them on. The manual is correct. The constants are only the fallback when no saved state exists.

## 6.3 "Similar" lockout limits are not the same

The manual says the other tools have similar limits. The code has three rules: a fixed 15 minutes (Dashboard, SysMon, 44helper), a delay that starts at the fifth failure and doubles, capped at 5 minutes (Node Talker, Node Recorder, pibrary, instmon, wifimon), and 44helper's 2-hour session, which differs from the 12-hour session of the others.

## 6.4 Tunnel watchdog and reboots

The tunnel section of wifimon has no reboot or power-off call. That matches the manual's "it never reboots". The only restarts it makes are of the tunnel service and the units that depend on it.

## 6.5 Emergency reset is the only Asterisk restart

Among the disconnect actions, only the Node Controller's emergency reset restarts Asterisk. Other disconnects send link commands only. Asterisk is restarted in other places, though: phone setup apply or revert (when a restart is requested), Node Talker's restart control, and SysMon's Restart Asterisk. Each of those restarts drops every link, so the operator should check the Dashboard before using one.

## 6.6 What was not checked in code

These items come from the manual and were not traced in the code for this summary: the Asterisk CLI key blocks, the masked-service and tunnel-kind locks in 44helper, SysMon's locked modules, quiet mode's tier order, the Keep-adapter rule, the hotspot, and Restore points. Trace them before the book states them as fact.